

For example: If no LinkedList instance has been created yet, write statements in the main program to create a mylist instance with the ordered inputs 1, 2, 3, 4, 5 to produce the output 1, 2, 3, 4, 5 by using the LinkedList class and Node class we have learned.

First of all, I suggest using a pencil or pen to underline or circle the important parts of the question. Ask yourself:

1. What does the question ask you to do?
2. What are the inputs, and what are the expected outputs?

Why is this solution wrong?:

```
mylist = LinkedList()
```

```
mylist.add(1)
```

```
mylist.add(2)
```

```
mylist.add(3)
```

```
mylist.add(4)
```

[เฉลยข้อ 1: การเขียนคำสั่งสร้าง LinkedList ด้วย method add()]:

* คำสั่ง Main Program เพื่อสร้าง mylist ด้วย input [1, 2, 3, 4, 5] ให้ได้ผลลัพธ์ 1, 2, 3, 4, 5:

```
mylist = LinkedList()
```

```
mylist.add(5) # แทรก 5 ที่หัวลิสต์ -> ได้ [5]
```

```
mylist.add(4) # แทรก 4 ที่หัวลิสต์ -> ได้ [4, 5]
```

```
mylist.add(3) # แทรก 3 ที่หัวลิสต์ -> ได้ [3, 4, 5]
```

```
mylist.add(2) # แทรก 2 ที่หัวลิสต์ -> ได้ [2, 3, 4, 5]
```

```
mylist.add(1) # แทรก 1 ที่หัวลิสต์ -> ได้ [1, 2, 3, 4, 5]
```

* หลักการสำคัญ: เมธอด add() จะแทรกข้อมูลใหม่ต่อท้าย Header (หัวแถวเสมอ)

ดังนั้นต้องป้อนข้อมูลจาก 'มากไปน้อย' (5 -> 1) เพื่อให้ได้ผลลัพธ์เรียงจากน้อยไปมาก

```
mylist.add(1)
```

```
mylist.insert(2,1)
```

```
mylist.insert(3,2)
```

For example: If an existing mylist instance already contains the input 3, write statements in the main program to update mylist with the ordered inputs 1, 2, 4, 5 so that the output becomes 1, 2, 3, 4, 5 by using the LinkedList and Node class we have learned.

The solution is:

mylist = LinkedList() **X** no need to create a new instance variable

mylist.add(1) #1 3

mylist.add(2) **X** Do not use the add method

[เฉลยข้อ 2: การอัปเดต LinkedList เดิมด้วย add() และ insert()]:

* กรณีไม่มี mylist ที่มีเลข 3 อยู่แล้ว และต้องการป้อน input [1, 2, 4, 5] ให้ได้ 1, 2, 3, 4, 5:

mylist.add(2) # mylist เดิมมี [3] แทรก 2 ที่หัว -> ได้ [2, 3]

mylist.add(1) # แทรก 1 ที่หัว -> ได้ [1, 2, 3]

mylist.insert(4, 3) # แทรก 4 ที่ตำแหน่ง index 3 -> ได้ [1, 2, 3, 4]

mylist.insert(5, 4) # แทรก 5 ที่ตำแหน่ง index 4 -> ได้ [1, 2, 3, 4, 5]

* ข้อควรระวังข้อสอบ:

1. ไม่ต้องสร้าง instance ซ้ำด้วย mylist = LinkedList() เพราะโจทย์ระบุว่ามียอยู่แล้ว
2. ห้ามใช้ mylist.insert(val, 0) เด็ดขาด เพราะ range(0) ไม่ทำงานใน Python ต้องใช้ add()
3. ค่า pos ใน insert(data, pos) ต้องสอดคล้องกับขนาดปัจจุบันของลิสต์

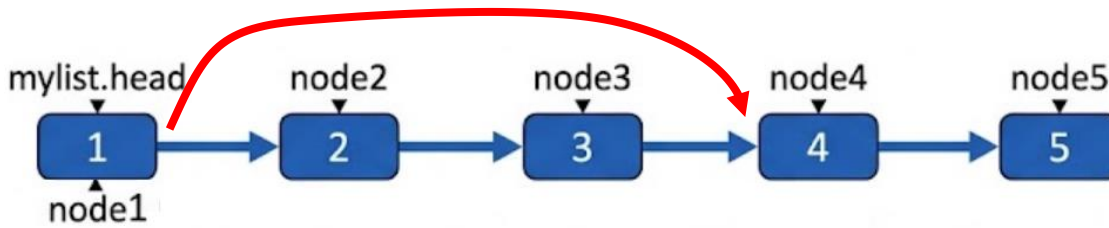
Th
Lin
nc
M
Th

1.

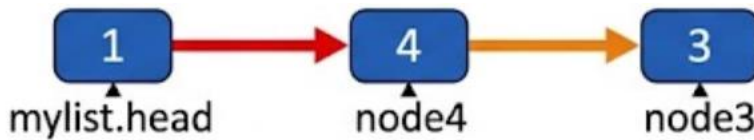
node1

5

2. `node1.next = node4`



3. `node4.next = node3`



4. `node3.next = node2`



[คำอธิบายกลไก Pointer Reconnection 5 ขั้นตอน]:

* สรุปขั้นตอนการสลับพอยน์เตอร์เพื่อย้ายตำแหน่งของ Node (Pointer Relinking):

1. `node1.next = node4` : ให้ Node 1 ชี้ไปที่ Node 4 (ข้าม Node 2, 3)

2. `node4.next = node3` : ให้ Node 4 ชี้อยากลับมาที่ Node 3

3. `node3.next = node2` : ให้ Node 3 ชี้ไปที่ Node 2

4. `node2.next = node5` : ให้ Node 2 ชี้ไปที่ Node 5 ตัวสุดท้าย

-> ผลลัพธ์ในหน่วยความจำ: 1 -> 4 -> 3 -> 2 -> 5

* กฎเหล็ก: การจัดการพอยน์เตอร์เป็นการเปลี่ยนโครงสร้างเชื่อมโยงจริง ไม่ใช่การคัดลอก .data

5.

n

แล้วมันต่างจาก การย้ายค่าใน `.data` ของโหนดอย่างไร เช่น

```
temp = node2.data
```

```
node2.data = node4.data
```

```
node4.data = temp
```

สอนแบบนี้เหมาะสม กับ วิชา **Data Structure** มั้ย ง่ายกว่าเยอะเลย

นักศึกษาเข้าใจผิดครับ

ทำไมต้องตัดต่อพอยเตอร์?

วิชา **Data Structure** ไม่ได้เน้นแค่ "ผลลัพธ์" แต่เน้นสอน "การจัดการหน่วยความจำ (Memory Manipulation)" ครับ เหตุผลหลัก ๆ มี 3 ข้อ:

เหตุผลที่ 1: ขนาดของข้อมูล (Data Size)

ในโจทย์เรียน ข้อมูลอาจจะมีแค่ตัวเลข 1, 2, 3 แต่ในชีวิตจริง `.data` ของโหนดหนึ่ง โหนด อาจจะเป็น "ประวัติลูกค้าทั้งคน" (ชื่อ, ที่อยู่, รูปถ่าย, ประวัติการซื้อ, อาร์เรย์ช้อนอาร์เรย์)

- **ถ้าสลับพอยเตอร์ (`.next`):** คอมพิวเตอร์แค่ย้ายเส้นด้ายที่ชี้ (ใช้พลังงานต่ำมาก เท่ากัน เสมอไม่ว่าข้อมูลจะใหญ่แค่ไหน)
- **ถ้าสลับข้อมูล (`.data`):** คอมพิวเตอร์ต้องคัดลอกข้อมูลขนาดมหาศาล ย้ายข้ามไปข้ามมา ในแรม ทำให้โปรแกรมทำงานช้าลงอย่างมากครับ

นั่นหมายถึงประสิทธิภาพของโค้ด

เหตุผลที่ 2: วัตถุประสงค์ของบทเรียน

Linked List ถูกสร้างขึ้นมาเพื่อแก้จุดอ่อนของ **Array** (ที่เวลาจะแทรกหรือลบต้องขยับข้อมูลตัวอื่น ๆ ทั้งหมด) จุดเด่นของมันคือ "ข้อมูลอยู่ที่เดิม แต่เปลี่ยนเส้นทางเดิน" อาจารย์จึงจำเป็นต้องบังคับให้เราเขียนสลับด้วย `.next` เพื่อฝึกสมองให้คิดเป็นระบบ ลำดับขั้นตอนก่อนหลัง

(Pointer Logic) ซึ่งเป็นพื้นฐานสำคัญมากในการไปเรียนโครงสร้างที่ยากกว่านี้ เช่น **Tree** (ต้นไม้) หรือ **Graph** (กราฟ) ที่ไม่มีทางโกงด้วยการสลับ `.data` ได้เลยครับ

ดังนั้น ถ้าเป็นตอนสอบ นักศึกษาทำด้วยการสลับ `.data` ถือว่าผิดวัตถุประสงค์ได้ระบุไว้ในโจทย์ด้วย จะได้ 0 คะแนน